MBLP131 · HAFTA 14

İşletim Sistemleri Temelleri

Hafta 14 · Kabuk Betiklerine Giriş

Bir kez yazın, kaydedin, bin kez çalıştırın.

Elektronik Haberleşme Teknolojisi Programı · Meslek Yüksekokulu · İstanbul Okan Üniversitesi

GEÇEN HAFTA

Dört Fikirde 13. Hafta

Süreçler

Her komut, PID'si ve ebeveyni (PPID) olan bir süreç olarak çalışır. ps, pstree ve top bunları gösterir; STAT sütunu R, S, T, Z durumlarını verir.

İş denetimi

komut & arka planda çalışır; Ctrl+Z bir işi durdurur; jobs, fg ve bg işleri ön plan ile arka plan arasında taşır.

Sinyaller

kill bir sinyal gönderir: SIGTERM (15) kibarca ister, SIGKILL (9) reddedilemez, Ctrl+C SIGINT (2) gönderir.

Öncelik

nice ve renice nice değerini (−20 … 19) ayarlar; değeri yalnızca root düşürebilir (yani önceliği yükseltebilir).

Bugün: betik de yalnızca bir programdır. Onu çalıştırdığınızda kabuk onun için yeni bir süreç oluşturur — 13. haftada gördüğümüz her şey burada da geçerlidir.

Bugünün Gündemi

1. **İlk betiğiniz** Neden otomasyon; shebang satırı; chmod +x; ./script.sh ile bash script.sh farkı.
2. **Değişkenler ve girdi** ad=değer, tırnaklar, $( ), $(( )), $1 $# $@, read.
3. **Kararlar** Çıkış durumu $?, test ve [ ], if / elif / else, bash -x ile hata ayıklama.
4. **Döngüler ve fonksiyonlar** for (listeler, {1..5}, dosyalar), while, fonksiyonlar.
5. **İki gerçek betik** backup.sh (zaman damgalı tar.gz) ve report.sh (günlük ve not özeti).
6. **Laboratuvar** Sık yapılan hatalar, ardından notlandırılan 3–4 betiği siz yazıyorsunuz.

BÖLÜM 1

İlk Betiğiniz

Kabuk betiği, içi komutlarla dolu bir metin dosyasıdır.

Neden Betik Yazarız?

Kabuk betiği (shell script), kabuk komutları içeren düz bir metin dosyasıdır. Kabuk dosyayı okur ve komutları birbiri ardına çalıştırır.

Komutları elle yazmak

- Her sabah: yedekleme ve günlük denetimi için 6 komut.
- Bir adımı unutmak ya da bir yolu yanlış yazmak kolay.
- Bir iş arkadaşınıza tam olarak ne yaptığınızı anlatmak zor.
- Siz uyurken saat 03.00'te çalışamaz.

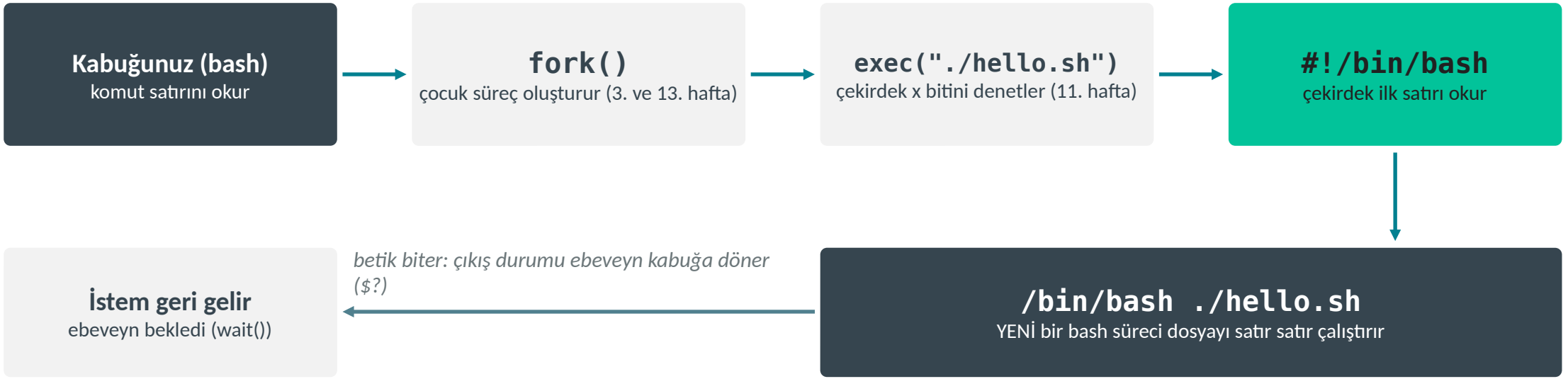
Komutları bir betiğe koymak

- Tek komut: ./backup.sh — her seferinde aynı adımlar.
- Bir hatayı bir kez düzeltirsiniz; düzeltilmiş kalır.
- Betik, yordamın belgesidir (yorumlar!).
- Bir zamanlayıcı (cron) onu kendiliğinden başlatabilir.

Pratik kural: aynı komut dizisini üçüncü kez yazıyorsanız bir betik yazın. Sistem yöneticileri, DevOps ve gömülü Linux mühendisleri bunu her gün yapar.

PERDE ARKASI

./hello.sh Yazdığınızda Ne Olur?



Sonuçları

- Betik bir çocuk süreçte çalışır: betiğin içindeki bir cd ya da değişken ataması terminalinizi değiştirmez.
- x izni yoksa → exec başarısız olur → "Permission denied".

Shebang satırı #!

- Dosyanın en ilk satırı olmalıdır: #! + yorumlayıcının mutlak yolu.
- #!/bin/bash bash içindir; #!/usr/bin/python3 aynı yolla Python çalıştırırdı.

hello.sh'nin Anatomisi

```
#!/bin/bash
# hello.sh -- my first shell script
# Run: bash hello.sh   or: chmod +x hello.sh; ./hello.sh
echo "Hello from a shell script!"
echo "I am running as: $(whoami)"
echo "Current folder:  $(pwd)"
```

1. Shebang: bu dosyayı hangi program çalıştırır
2. Yorumlar: bash yok sayar, insanlar okur
3. Komutlar: tam olarak istemde yazacağınız gibi

Bir metin düzenleyiciyle oluşturun

- **nano hello.sh** — satırları yazın, Ctrl+O ve Enter ile kaydedin, Ctrl+X ile çıkın. (Her düzenleyici olur: vim, gedit, VS Code.)
- **$(whoami)** komut yerine koymadır (command substitution): bash whoami komutunu çalıştırır ve çıktısını metnin içine yerleştirir (2. bölüm).
- **Ad:** .sh uzantısı yalnızca insanlar için bir alışkanlıktır; Linux uzantıya değil, x bitine ve shebang satırına bakarak karar verir (7. hafta: dosya türleri).

CANLI GÖSTERİM

Çalıştırma: bash hello.sh ile ./hello.sh

```
student@okan-lab:~/lab/scripts$ bash hello.sh
Hello from a shell script!
I am running as: student
Current folder:  /home/student/lab/scripts
student@okan-lab:~/lab/scripts$ ./hello.sh
bash: ./hello.sh: Permission denied
student@okan-lab:~/lab/scripts$ echo $?
126
student@okan-lab:~/lab/scripts$ ls -l hello.sh
-rw-rw-r-- 1 student student 204 Oct 12 17:18 hello.sh
student@okan-lab:~/lab/scripts$ chmod +x hello.sh
student@okan-lab:~/lab/scripts$ ls -l hello.sh
-rwxrwxr-x 1 student student 204 Oct 12 17:18 hello.sh
student@okan-lab:~/lab/scripts$ ./hello.sh
Hello from a shell script!
I am running as: student
Current folder:  /home/student/lab/scripts
student@okan-lab:~/lab/scripts$ hello.sh
bash: hello.sh: command not found
```

Nelere dikkat edin

- **bash hello.sh** x izni olmadan da çalışır: bash dosyayı yalnızca okur.
- **./hello.sh** x bitine ihtiyaç duyar: Permission denied, çıkış durumu 126.
- **chmod +x** kullanıcı, grup ve diğerlerine x ekler (11. hafta): rw- → rwx.
- **hello.sh** tek başına: bulunamaz, çünkü geçerli dizin $PATH içinde değildir. Bu yüzden ./

Bir Betiği Çalıştırmanın Dört Yolu

Komut	Gerekenler	Nerede çalışır	Tipik kullanım
bash hello.sh	r izni	yeni bir bash süreci	hızlı deneme; x biti olmayan betik
./hello.sh	r + x, shebang satırı	yeni süreç (yorumlayıcı #! satırından)	kendi betiğinizi çalıştırmanın olağan yolu
hello.sh	r + x, dizin $PATH içinde	yeni süreç	kurulu araçlar, ör. ~/bin içinde
source hello.sh (ya da . hello.sh)	r izni	GEÇERLİ kabuk	değişken ayarlayan dosyalar, ör. ~/.bashrc

Bu derste ./script.sh kullanın. bash -n script.sh hiçbir şey çalıştırmadan sözdizimini denetler; bash -x script.sh her komutu çalışırken gösterir (3. bölüm).

BÖLÜM 2

Değişkenler, Tırnaklar ve Girdi

Değer saklamak, metin oluşturmak, argümanları ve klavye girdisini okumak.

Değişkenler: ad=değer, Boşluk Yok!

```
student@okan-lab:~/lab/scripts$ name="Ayse"
student@okan-lab:~/lab/scripts$ echo $name
Ayse
student@okan-lab:~/lab/scripts$ echo "Hello, $name!"
Hello, Ayse!
student@okan-lab:~/lab/scripts$ name = Ayse
bash: name: command not found
```

name = Ayse şöyle okunur: name komutunu = ve Ayse argümanlarıyla çalıştır. Bu yüzden "name: command not found". Bash'te boşluk süs değildir — sözcükleri ayırır.

Kurallar

- **Atama:** ad=değer — = işaretinin çevresinde boşluk yok.
- **Kullanım:** $ad ya da ${ad}; tanımsız bir değişken yalnızca boştur.
- **Adlar:** harf, rakam, _; rakamla başlamaz; büyük/küçük harfe duyarlıdır (name ≠ NAME).
- **Her şey metindir:** count=5, "5" metnini saklar; aritmetik için $(( )) gerekir.
- **Gelenek:** ortam değişkenleri BÜYÜK harfle (HOME, USER, PATH); kendi değişkenleriniz küçük harfle.

Komut Yerine Koyma $( ) ve Aritmetik $(( ))

```
name="Ayse"                      # NO spaces around the = sign
course=MCMP117
week=14
echo "Student: $name, course: $course, week $week"

today=$(date +%Y-%m-%d)          # $( ) = the OUTPUT of a command
lines=$(wc -l < ~/lab/docs/syllabus.txt)
echo "Today is $today; the syllabus has $lines lines"

left=$((15 - week))              # $(( )) = integer arithmetic
echo "Weeks left until the final exam: $left"
echo "7 / 2 = $((7 / 2)) remainder $((7 % 2))   (integers only!)"
```

```
student@okan-lab:~/lab/scripts$ ./variables.sh
Student: Ayse, course: MCMP117, week 14
Today is 2026-10-12; the syllabus has 15 lines
Weeks left until the final exam: 1
7 / 2 = 3 remainder 1   (integers only!)
```

İki genişletme

- **$(komut)** → komutun çıktısıyla değiştirilir (sondaki satır sonu atılır). Eski biçim: `komut`.
- **$((ifade))** → tamsayı aritmetiği: + − * / % ve ( ). İçeride adların önündeki $ isteğe bağlıdır.
- **Yalnızca tamsayı:** 7 / 2 = 3, kalan 7 % 2 = 1. Ondalık için awk (report.sh) ya da bc kullanın.

Çift Tırnak, Tek Tırnak, Tırnaksız

```
name="Ayse"
file="final report.txt"          # a file name that contains a space
# shellcheck disable=SC2016      # (on purpose)
echo 'Single: Hello $name'       # '...' : everything is literal
echo "Double: Hello $name"       # "..." : $var and $( ) expanded
echo "Escaped: it costs \$5"     # \$ : one literal dollar sign
cd ~/lab/docs || exit 1
wc -l "$file"                    # quoted:   ONE argument
# shellcheck disable=SC2086      # (on purpose)
wc -l $file                      # unquoted: TWO arguments
```

```
student@okan-lab:~/lab/scripts$ ./quotes.sh
Single: Hello $name
Double: Hello Ayse
Escaped: it costs $5
2 final report.txt
wc: final: No such file or directory
wc: report.txt: No such file or directory
0 total
```

Tırnaklar ne yapar?

- **"çift":** boşlukları korur; $değişken ve $( ) yine genişletilir. Neredeyse her zaman bunu kullanın.
- **'tek':** her şey olduğu gibi kalır, hiçbir genişletme yok.
- **\ (ters eğik çizgi):** tek bir karakteri olduğu gibi bırakır: \$, \".
- **Tırnaksız:** değer boşluklardan bölünür (ve * genişletilir). "final report.txt" iki argüman olur — iki hata.

Konumsal Parametreler: $0, $1 … $9, $#, $@

```
echo "Script name  \$0 = $0"
echo "First arg    \$1 = $1"
echo "Second arg   \$2 = $2"
echo "Arg count    \$# = $#"
echo "All args     \$@ =" "$@"
i=1
for arg in "$@"; do             # "green tea" stays ONE word
    echo "  argument $i: [$arg]"
    i=$((i + 1))
done
```

Değişken	Anlamı
`$0`	betiğin, yazıldığı biçimdeki adı
`$1 … $9`	1. … 9. argüman (10. için ${10})
`$#`	argüman sayısı
`"$@"`	tüm argümanlar, her biri ayrı bir sözcük
`$?`	son komutun çıkış durumu (3. bölüm)
`$$`	bu kabuğun PID'si (13. hafta)

```
student@okan-lab:~/lab/scripts$ ./args.sh apple "green tea" 42
Script name  $0 = ./args.sh
First arg    $1 = apple
Second arg   $2 = green tea
Arg count    $# = 3
All args     $@ = apple green tea 42
  argument 1: [apple]
  argument 2: [green tea]
  argument 3: [42]
```

Kabuk, betik başlamadan önce komut satırını sözcüklere ayırır: tırnaklar sayesinde "green tea" TEK argümandır.

read ile Klavyeden Girdi Okuma

```
read -r -p "What is your name? " name
read -r -p "Which year were you born? " year
age=$(( $(date +%Y) - year ))
echo "Nice to meet you, $name. You are about $age years old."
```

```
student@okan-lab:~/lab/scripts$ ./greet.sh
What is your name? Ayse
Which year were you born? 2006
Nice to meet you, Ayse. You are about 20 years old.
```

Tek satırda read

- **read -r var** bir satır bekler ve onu var içinde saklar.
- **-p "metin"** önce bir istem gösterir.
- **-r** ters eğik çizgileri yazıldığı gibi tutar (her zaman kullanın).

Argümanlar ($1) mı, read mi? Otomasyon için argümanlar daha iyidir: ./grade.sh 72, klavye başında kimse yokken başka bir betikten ya da cron'dan çalışabilir. read ise etkileşimli araçlar içindir, ör. bir menü ya da oyun (Lab alıştırması 14).

ÖZET TABLO

Şimdiye Kadarki Sözdizimi

Siz yazarsınız	Bash şunu yapar	Örnek (gerçek çıktı)
#!/bin/bash	ilk satır: yorumlayıcıyı seçer	./hello.sh, /bin/bash ile çalışır
name=value	atama (boşluk yok!)	week=14
"$name" '$name'	genişletir / olduğu gibi bırakır	Hello Ayse / Hello $name
$(command)	bir komutun çıktısını yerleştirir	$(date +%Y-%m-%d) → 2026-10-12
$((expression))	tamsayı aritmetiği	$((3 + 4 * 2)) → 11
$1 $# "$@"	argümanlar, sayıları, hepsi	./args.sh a b → $# = 2
read -r -p "? " x	klavyeden bir satır okur	x=Ayse

ARA

10 Dakika Mola

Sırada: kararlar, döngüler, fonksiyonlar — ve iki gerçek otomasyon betiği.

BÖLÜM 3

Karar Verme

Her komut bir evet/hayır sorusunu yanıtlar: çıkış durumu.

Her Komut Bir Çıkış Durumu Döndürür: $?

```
student@okan-lab:~/lab/scripts$ ls ../docs > /dev/null
student@okan-lab:~/lab/scripts$ echo $?
0
student@okan-lab:~/lab/scripts$ ls ../nothing
ls: cannot access '../nothing': No such file or directory
student@okan-lab:~/lab/scripts$ echo $?
2
student@okan-lab:~/lab/scripts$ nosuchcommand
bash: nosuchcommand: command not found
student@okan-lab:~/lab/scripts$ echo $?
127
student@okan-lab:~/lab/scripts$ grep -q ERROR ../logs/system.log
student@okan-lab:~/lab/scripts$ echo $?
0
student@okan-lab:~/lab/scripts$ grep -q PANIC ../logs/system.log
student@okan-lab:~/lab/scripts$ echo $?
1
```

$?	Anlamı
0	başarı ("doğru")
1	genel hata / "yanlış"
2	yanlış kullanım (ls, grep: hata)
126	bulundu ama çalıştırılamıyor
127	komut bulunamadı

0 başarı demektir — 0'ın yanlış olduğu C'nin tam tersi! Bir program bunu exit(n) ya da main'den return n ile ayarlar (3. hafta); bir betik exit n ile. Aralık 0–255.

test ve [ ]: Karşılaştırma Yapan Bir Komut

```
student@okan-lab:~/lab/scripts$ [ 5 -gt 3 ]
student@okan-lab:~/lab/scripts$ echo $?
0
student@okan-lab:~/lab/scripts$ [ "abc" = "abd" ]
student@okan-lab:~/lab/scripts$ echo $?
1
student@okan-lab:~/lab/scripts$ type [
[ is a shell builtin
```

Tür	İşleçler	Örnek
Sayılar	`-eq -ne -lt -le -gt -ge`	`[ "$n" -ge 50 ]`
Dizgiler	`= != -z(boş) -n(dolu)`	`[ "$ans" = yes ]`
Dosyalar	`-e var -f dosya -d dizin`	`[ -d ~/lab/backup ]`
	`-r -w -x (r/w/x izni)`	`[ -x backup.sh ]`
Mantık	`! (değil)  &&  ||`	`[ -f a ] && [ -f b ]`

[bir komuttur: boşluklar önemli

- [5 -gt 3], test 5 -gt 3 ile aynıdır. Son argüman] olmalıdır.
- Her parça ayrı bir sözcüktür: ["$n" -gt 3]. Boşluk olmazsa [$n bir komut adına dönüşür: "[5: command not found".
- [ ] içinde değişkenleri tırnaklayın: boş bir $x kaybolur ve testi bozar.

Sayılar: -lt, -gt … (harfler!). Dizgiler: = ve !=. [ ] içinde < ve > karşılaştırma değil, yönlendirmedir (12. hafta).

if: Komutları Yalnızca Test Başarılıysa Çalıştırmak

```
if [ $# -ne 1 ]; then                    # exactly one argument?
    echo "Usage: $0 PATH" >&2            # >&2 : message goes to stderr
    exit 2                               # 2 = wrong usage
fi
path=$1
if [ -f "$path" ]; then
    echo "$path: regular file, $(wc -l < "$path") lines"
elif [ -d "$path" ]; then
    echo "$path: directory"
elif [ -e "$path" ]; then
    echo "$path: exists, but is neither a file nor a directory"
else
    echo "$path: does not exist" >&2
    exit 1                               # 1 = failure
fi
exit 0                                   # 0 = success
```

Yapı

- if TEST; then … elif TEST; then … else … fi
- TEST herhangi bir komut olabilir: [ ], grep -q, mkdir …
- fi (tersten if) bloğu bitirir. Gövdeyi 4 boşluk içeri alın.

Buradaki iyi alışkanlıklar

- Önce $# denetlenir; bir Usage satırı basılır.
- Hatalar stderr'e gider: >&2 (12. hafta).
- Ayrı çıkış kodları: 0 tamam, 1 hata, 2 kullanım.

CANLI GÖSTERİM

check.sh: Dört Tür Yol, Üç Çıkış Kodu

```
student@okan-lab:~/lab/scripts$ ./check.sh ../docs/syllabus.txt
../docs/syllabus.txt: regular file, 15 lines
student@okan-lab:~/lab/scripts$ ./check.sh ../logs
../logs: directory
student@okan-lab:~/lab/scripts$ ./check.sh /dev/null
/dev/null: exists, but is neither a file nor a directory
student@okan-lab:~/lab/scripts$ ./check.sh nothing.txt
nothing.txt: does not exist
student@okan-lab:~/lab/scripts$ echo $?
1
student@okan-lab:~/lab/scripts$ ./check.sh
Usage: ./check.sh PATH
student@okan-lab:~/lab/scripts$ echo $?
2
```

Nelere dikkat edin

- syllabus.txt: -f doğrudur, bu yüzden ilk dal çalışır.
- ../logs: -f yanlış, -d doğru: elif dalı.
- /dev/null: vardır (-e), ama bir aygıt dosyasıdır (9. hafta: /dev); bu yüzden zincir üçüncü dala ulaşır.
- Olmayan dosya: exit 1. Argüman yok: Usage ve exit 2.

grade.sh: Sayılarla if / elif / else

```
score=$1
if [ -z "$score" ]; then               # -z : the string is empty
    echo "Usage: $0 SCORE (0-100)" >&2
    exit 2
fi
if [ "$score" -lt 0 ] || [ "$score" -gt 100 ]; then
    echo "Error: score must be between 0 and 100" >&2
    exit 1
elif [ "$score" -ge 85 ]; then
    letter=A
elif [ "$score" -ge 70 ]; then
    letter=B
elif [ "$score" -ge 50 ]; then
    letter=C
else
    letter=F
fi
echo "Score $score -> grade $letter"
```

```
student@okan-lab:~/lab/scripts$ ./grade.sh 92
Score 92 -> grade A
student@okan-lab:~/lab/scripts$ ./grade.sh 72
Score 72 -> grade B
student@okan-lab:~/lab/scripts$ ./grade.sh 49
Score 49 -> grade F
student@okan-lab:~/lab/scripts$ ./grade.sh 150
Error: score must be between 0 and 100
student@okan-lab:~/lab/scripts$ ./grade.sh
Usage: ./grade.sh SCORE (0-100)
```

Bir merdiven gibi okuyun

- bash testleri yukarıdan aşağıya dener ve yalnızca testi doğru olan İLK dalı çalıştırır.
- `||` = veya: ikinci test yalnızca ilki yanlışsa çalışır.

Hata Ayıklama: bash -n, bash -x ve Hatalı Girdi

```
student@okan-lab:~/lab/scripts$ bash -x grade.sh 72
+ score=72
+ '[' -z 72 ']'
+ '[' 72 -lt 0 ']'
+ '[' 72 -gt 100 ']'
+ '[' 72 -ge 85 ']'
+ '[' 72 -ge 70 ']'
+ letter=B
+ echo 'Score 72 -> grade B'
Score 72 -> grade B
```

```
student@okan-lab:~/lab/scripts$ ./grade.sh abc
./grade.sh: line 9: [: abc: integer expression expected
./grade.sh: line 9: [: abc: integer expression expected
./grade.sh: line 12: [: abc: integer expression expected
./grade.sh: line 14: [: abc: integer expression expected
./grade.sh: line 16: [: abc: integer expression expected
Score abc -> grade F
```

Üç araç

- **bash -n script.sh:** yalnızca sözdizimi denetimi; hiçbir şey çalışmaz.
- **bash -x script.sh:** her komutu çalıştırmadan önce (+ işaretiyle ve değerler yerine konmuş olarak) yazdırır. Betik içinde: izlemek istediğiniz kısmı set -x … set +x arasına alın.
- **Girdiyi doğrulayın:** grade.sh abc hatalar basar ama yine de grade F der — bir hata (bug). Önce girdiyi denetleyin (lab alıştırması fikri).

BÖLÜM 4

Döngüler ve Fonksiyonlar

Her öğe için ya da bir koşul sürdükçe bir şey yapmak.

for: Listedeki Her Öğe İçin Bir Kez

```
for city in Istanbul Ankara Izmir; do       # 1) over a list of words
    echo "City: $city"
done

for i in {1..5}; do                          # 2) over a range of numbers
    echo -n "$i squared is $((i * i)); "
done
echo                                         # finish the line

for f in ~/lab/docs/*.txt; do                # 3) over files (wildcard)
    echo "$(basename "$f"): $(wc -l < "$f") lines"
done
```

```
student@okan-lab:~/lab/scripts$ ./loops.sh
City: Istanbul
City: Ankara
City: Izmir
1 squared is 1; 2 squared is 4; ...
final report.txt: 2 lines
linux_quotes.txt: 5 lines
notes_01.txt: 1 lines
notes_02.txt: 1 lines
notes_03.txt: 1 lines
syllabus.txt: 15 lines
```

for x in LİSTE; do ... done

- **Sözcük listesi:** tek tek yazılır ya da "$@".
- **{1..5}:** küme parantezi genişletmesi → 1 2 3 4 5 (10. hafta).
- ***.txt:** kabuk joker karakteri eşleşen dosya adlarına genişletir — "$f" yazarsanız boşluklu adlarda bile güvenlidir.
- **Ayrıca:** liste olarak $(komut), ör. for u in $(cut -d, -f3 file).

while: Test Başarılı Olduğu Sürece Tekrarla

```
n=3
while [ "$n" -gt 0 ]; do                    # repeat WHILE the test is true
    echo "Countdown: $n"
    n=$((n - 1))
done
echo "Go!"

sum=0
count=0
while read -r x; do                         # one line of the file per turn
    sum=$((sum + x))
    count=$((count + 1))
done < ~/lab/data/numbers.txt               # the whole loop reads this file
echo "$count numbers, sum = $sum, average = $((sum / count)) (integer)"
```

```
student@okan-lab:~/lab/scripts$ ./while.sh
Countdown: 3
Countdown: 2
Countdown: 1
Go!
20 numbers, sum = 11325, average = 566 (integer)
```

İki klasik kalıp

- **Sayaç:** değişkeni döngünün içinde değiştirin — n=$((n - 1)) satırını unutursanız döngü hiç bitmez (Ctrl+C, 13. hafta).
- **Dosya okuma:** while read -r line; do … done < file her turda bir satır okur; dosya sonunda read başarısız olur ve döngü durur.
- **Sağlama:** 11325 / 20 = 566,25; bash 566 yazar.

Fonksiyonlar: Bir Komut Bloğuna Ad Vermek

```
line() {                                    # no arguments
    echo "-----------------------------------"
}
count_word() {                              # $1 = word, $2 = file
    local n                                 # local: exists only in here
    n=$(grep -c -- "$1" "$2")
    echo "$1 appears on $n lines of $(basename "$2")"
}
is_even() {                                 # the exit status is the answer
    [ $(( $1 % 2 )) -eq 0 ]
}
line
count_word ERROR ~/lab/logs/system.log
count_word WARN  ~/lab/logs/system.log
line
for k in 7 12; do
    if is_even "$k"; then echo "$k is even"; else echo "$k is odd"; fi
done
```

```
student@okan-lab:~/lab/scripts$ ./functions.sh
---------------------------
ERROR appears on 13 lines of system.log
WARN appears on 28 lines of system.log
---------------------------
7 is odd
12 is even
```

Nasıl çalışırlar?

- Kullanmadan önce tanımlayın: ad() { … }
- Bir komut gibi çağırın: count_word WARN file. İçeride $1 $2 fonksiyonun kendi argümanlarıdır.
- local bir değişkeni fonksiyona özel tutar. Son komutun çıkış durumu fonksiyonun yanıtıdır.

ÖZET TABLO

Bir Bakışta Denetim Yapıları

Yapı	Sözdizimi (tek satır)	Bitiş
if	if ["$n" -gt 0]; then …; elif …; then …; else …; fi	fi
for (liste)	for f in *.txt; do echo "$f"; done	done
for (aralık)	for i in {1..5}; do echo "$i"; done	done
while	while ["$n" -gt 0]; do n=$((n - 1)); done	done
dosya okuma	while read -r line; do …; done < file.txt	done
fonksiyon	name() { local x=$1; echo "$x"; }	}
ve / veya	mkdir -p dir && echo ok; [-f f] || exit 1	—

BÖLÜM 5

İki Gerçek Otomasyon Betiği

~/lab/scripts içinde backup.sh ve report.sh

backup.sh: docs/ Klasörünün Zaman Damgalı Arşivi

```
lab="$HOME/lab"
src="docs"                                  # folder to save (inside $lab)
dest="$lab/backup"
stamp=$(date +%Y%m%d_%H%M%S)                # e.g. 20261012_143005
archive="$dest/docs_$stamp.tar.gz"

if [ ! -d "$lab/$src" ]; then
    echo "Error: $lab/$src not found" >&2
    exit 1
fi
mkdir -p "$dest"                            # no error if it exists
if tar -czf "$archive" -C "$lab" "$src"; then
    n=$(tar -tzf "$archive" | grep -vc '/$')    # count files, not folders
    echo "Backup created: $archive"
    echo "Files: $n, size: $(du -h "$archive" | cut -f1)"
else
    echo "Error: tar failed" >&2
    exit 1
fi
```

Satır satır

- **date +%Y%m%d_%H%M%S** benzersiz ve sıralanabilir bir ad verir.
- **[! -d ...]:** kaynak yoksa erkenden durur.
- **tar -czf A -C dir docs:** gzip ile sıkıştırılmış (z) bir arşiv dosyası (f) oluşturur (c); -C = önce ~/lab'a geç, böylece yollar docs/ ile başlar.
- **if tar ...:** tar'ın kendi çıkış durumunu test eder.

CANLI GÖSTERİM

backup.sh İş Başında

```
student@okan-lab:~/lab/scripts$ ./backup.sh
Backup created: /home/student/lab/backup/docs_20261012_085013.tar.gz
Files: 7, size: 4.0K
student@okan-lab:~/lab/scripts$ ls -l ../backup
total 4
-rw-rw-r-- 1 student student 862 Oct 12 08:50 docs_20261012_085013.tar.gz
student@okan-lab:~/lab/scripts$ tar -tzf ../backup/docs_*.tar.gz
docs/
docs/notes_02.txt
docs/syllabus.txt
docs/.settings
docs/notes_01.txt
docs/notes_03.txt
docs/linux_quotes.txt
docs/final report.txt
student@okan-lab:~/lab/scripts$ ./backup.sh
Backup created: /home/student/lab/backup/docs_20261012_085015.tar.gz
Files: 7, size: 4.0K
student@okan-lab:~/lab/scripts$ ls ../backup
docs_20261012_085013.tar.gz  docs_20261012_085015.tar.gz
```

Nelere dikkat edin

- Her çalıştırma yeni bir dosya oluşturur: zaman damgası adın parçasıdır, hiçbir şeyin üzerine yazılmaz.
- Files: 7 — altı görünür belge ve gizli .settings (tar gizli dosyaları da alır).
- "final report.txt" doğru arşivlenir: betik her değişkeni tırnaklar.
- Tarih yıl önce yazıldığı için adlar zamana göre sıralanır.

report.sh: Bir Günlük Dosyasını ve CSV'yi Özetlemek

```
log="$HOME/lab/logs/access.log"
csv="$HOME/lab/data/students.csv"
for f in "$log" "$csv"; do
    if [ ! -r "$f" ]; then echo "Error: cannot read $f" >&2; exit 1; fi
done
echo "===== Lab report, $(date '+%Y-%m-%d %H:%M') ====="

echo "--- Web server: $(basename "$log") ---"
total=$(wc -l < "$log")
errors=$(awk '$9 >= 400' "$log" | wc -l)    # field 9 = status code
echo "Requests: $total, errors (4xx/5xx): $errors ($((100 * errors / total))%)"
echo "Top 3 pages:"
awk '{print $7}' "$log" | sort | uniq -c | sort -rn | head -3

echo "--- Students: $(basename "$csv") ---"
echo "Students: $(tail -n +2 "$csv" | wc -l)"
echo "Per department:"
tail -n +2 "$csv" | cut -d, -f3 | sort | uniq -c
awk -F, 'NR > 1 { m += $4; f += $5; n++ }
         END { printf "Average midterm %.1f, average final %.1f\n", m/n, f/n }' "$csv"
passed=$(awk -F, 'NR > 1 && 0.4*$4 + 0.6*$5 >= 50' "$csv" | wc -l)
echo "Passed (0.4 x midterm + 0.6 x final >= 50): $passed"
```

CANLI GÖSTERİM

report.sh: Gerçek Çıktı, Sağlanmış

```
student@okan-lab:~/lab/scripts$ ./report.sh
===== Lab report, 2026-10-12 08:50 =====
--- Web server: access.log ---
Requests: 300, errors (4xx/5xx): 98 (32%)
Top 3 pages:
     41 /courses/mcmp117
     37 /about.html
     33 /index.html
--- Students: students.csv ---
Students: 40
Per department:
     10 CS
     17 ECT
      2 EE
     11 ME
Average midterm 67.0, average final 65.6
Passed (0.4 x midterm + 0.6 x final >= 50): 35
```

Sayılar doğrulandı (Python)

- **Hatalar:** 300 isteğin 98'inin durum kodu ≥ 400; 100·98/300 = 32,67, bash tamsayı aritmetiği 32 yazar.
- **Eşitlik:** /index.html ve /images/logo.png 33'er istek aldı; sırayı sort belirler.
- **Bölümler:** 17 + 11 + 10 + 2 = 40.
- **Ortalamalar:** vize 66,95 → 67.0 yazılır; final 65,6.
- **Geçenler:** 40 öğrencinin 35'i için 0,4m + 0,6f ≥ 50.

Yapacağınız Hatalar (Herkes Yapar)

= çevresinde boşluk

```
name = "Ayse"
echo "Hello $name"
$ ./bad_space.sh
./bad_space.sh: line 2: name: command
not found
Hello
```

Düzeltme: name="Ayse"

[ ] içinde boşluk yok

```
n=5
if [$n -gt 3]; then echo "big"; fi
$ ./bad_test.sh
./bad_test.sh: line 3: [5: command
not found
```

Düzeltme: if ["$n" -gt 3]; then

Eksik tırnaklar

```
answer=""
if [ $answer = yes ]; then echo "OK";
fi
$ ./bad_quotes.sh
./bad_quotes.sh: line 3: [: =: unary
operator expected
```

Düzeltme: ["$answer" = yes]

Hata iletisini okuyun: betiğin adını, satır numarasını ve bash'in anlayamadığı sözcüğü söyler.

Windows Satır Sonları, Eksik fi, Eksik chmod

```
student@okan-lab:~/lab/scripts$ ./win.sh
bash: ./win.sh: cannot execute: required file not found
student@okan-lab:~/lab/scripts$ bash win.sh
win.sh: line 2: cd: $'/home/student/lab\r': No such file or directory
Hello from Windows
student@okan-lab:~/lab/scripts$ cat -A win.sh
#!/bin/bash^M$
cd ~/lab^M$
echo "Hello from Windows"^M$
student@okan-lab:~/lab/scripts$ sed -i 's/\r$//' win.sh
student@okan-lab:~/lab/scripts$ ./win.sh
Hello from Windows
```

```
student@okan-lab:~/lab/scripts$ bash -n bad_if.sh
bad_if.sh: line 4: syntax error: unexpected end of file
```

CRLF: görünmez hata

- Windows düzenleyicileri satırları \r\n ile bitirir; Linux \n bekler. \r (^M olarak görünür) sözcüklere yapışır: çekirdek "/bin/bash\r" adlı bir yorumlayıcı arar.
- Yeni bash: "cannot execute: required file not found"; eski bash (ör. Ubuntu 22.04): "/bin/bash^M: bad interpreter".
- Düzeltme: sed -i 's/\r$//' file ya da dos2unix file; VS Code'da LF seçin.
- chmod +x unutulursa → Permission denied (slayt 8).

LAB · NOTLU

Sınıf İçi Lab: Dört Betik Yazın (35 Dakika)

1	`power.sh N`	N, N^2 ve N^3 değerlerini yazdırın. Argüman yoksa: bir Usage satırı ve exit 2.	*Alış. 2*
2	`what.sh YOL…`	Her argüman için dosya (satır sayısıyla), dizin ya da yok yazın; biri yoksa exit 1.	*Alış. 6*
3	`levels.sh`	~/lab/logs/system.log içindeki INFO, WARN ve ERROR satırlarını bir for döngüsü ve grep -c ile sayın.	*Alış. 11*
4	`backup_keep.sh`	Zorlu: backup.sh'yi yalnızca en yeni 3 arşiv kalacak biçimde genişletin.	*Alış. 13*

Notlandırma: hatasız çalışma (%40), doğru çıktı (%30), hatalı girdiyi ele alma (%15), yorumlar + tırnaklar + shebang (%15). Her betiği önce bash -n ile deneyin. Dört .sh dosyasını dersin Telegram kanalında duyurulan biçimde teslim edin.

KISA SINAV

Kendinizi Sınayın

1. ./hello.sh neden "Permission denied" verirken bash hello.sh çalışır? Hangi çıkış durumunu alırsınız?

2. x=7; y=2. echo "$((x / y))", echo '$x' ve echo "xy" ne yazdırır?

3. report.txt dosyası var olduğunda VE n değişkeni 10'dan büyük olduğunda doğru olan bir test yazın.

4. ./args.sh one "two three" four — $# ve $2 nedir?

14. Hafta: Akılda Kalması Gerekenler

- **Betik:** komutlardan oluşan bir metin dosyası; 1. satırda #!/bin/bash, chmod +x, ./script.sh ile çalıştırma. Bir çocuk süreçte çalışır (fork + exec).
- **Değişkenler:** boşluksuz ad=değer; çift tırnak içinde "$ad"; $( ) komut çıktısını yerleştirir; $(( )) tamsayı aritmetiği yapar.
- **Girdi:** argümanlar $1 … $9, $#, "$@"; klavye için read -r.
- **Kararlar:** her komutun bir çıkış durumu vardır (0 = başarı); [ ] dosyaları (-f, -d, -e), sayıları (-lt) ve dizgileri (=, -z) test eder; if / elif / else / fi.
- **Döngüler ve fonksiyonlar:** listeler, {1..5} ve *.txt üzerinde for; sayaçlı ya da dosya okuyan while; ad() { … }.
- **Hata ayıklama:** bash -n (sözdizimi), bash -x (izleme); hatadaki satır numarasını okuyun; CRLF satır sonlarına dikkat.

GELECEK HAFTA

Final Sınavı Tekrarı (15. hafta)

15. haftanın konuları:

- Dersin tamamı: kuram 1–7. haftalar ve Linux 9–14. haftalar
- Alıştırma soruları: zamanlama, bellek, sayfa değiştirme
- "Komutu / boru hattını / betiği yazın" soruları
- Cevap anahtarıyla birlikte örnek bir final sınav kâğıdı
- Sınav tarihi ve salonu: dersin Telegram kanalını takip edin

SORULAR?

Tartışma

Dersten sonra sorularınız olursa dersin Telegram kanalında sorun — yanıttan herkes yararlanır.

Dr. Öğr. Üyesi Hasan Oğuz